



**PROJETO FINAL
BIG-DATA**

GABRIEL ROGÉRIO APARECIDO PAES SILVA
GILSON DOS SANTOS JUNIOR
GUILHERME FABRICIO BRITO DA ROSA
GUSTAVO ALESSANDRO DE SOUZA SABINO

LAVRAS
2026

PROCESSAMENTO DE DADOS MASSIVOS

RELATÓRIO

Relatório apresentado à disciplina de Big-Data, do professor Vinicius Vitor, para obtenção de nota relativa ao período vigente.

LAVRAS
2026

RELATÓRIO

1. Contexto e Motivação

O Counter-Strike: Global Offensive (CS:GO) é um dos jogos de tiro em primeira pessoa mais populares do mundo, reunindo milhões de jogadores e gerando uma enorme quantidade de dados a cada partida. Durante um jogo competitivo são registrados milhares de eventos, como movimentação dos jogadores, utilização de armas, eliminações e eventos de dano.

Plataformas como a ESEA armazenam essas informações em grandes bases de dados, permitindo a realização de análises estatísticas sobre o desempenho dos jogadores e das partidas.

O objetivo deste projeto é utilizar o Apache Spark para processar um grande volume de eventos de dano extraídos de partidas competitivas de CS:GO, identificando padrões relacionados à eficiência das armas, ao desempenho dos jogadores e às vantagens estratégicas entre os lados Counter-Terrorist (CT) e Terrorist (T).

Este problema se enquadra no contexto de Big Data pelos seguintes motivos:

- O conjunto de dados possui aproximadamente seis milhões de registros.
- Os arquivos CSV ocupam mais de 1 GB de armazenamento.
- O conjunto completo é composto por diversas partes, totalizando vários gigabytes.
- O processamento envolve agregações e operações analíticas sobre milhões de linhas, tornando necessário o uso de processamento distribuído.

Dessa forma, foi escolhido o Apache Spark como framework de processamento distribuído, utilizando Docker para simular um cluster de execução.

2. Descrição do Conjunto de Dados

O conjunto de dados utilizado foi o **ESEA Master Damage Demos**, disponível publicamente na plataforma Kaggle.

Cada registro representa um evento de dano ocorrido durante uma partida competitiva de CS:GO.

Entre os principais atributos utilizados estão:

- Identificador da partida;
- Número da rodada;
- Arma utilizada;
- Categoria da arma;
- Dano causado à vida (HP);
- Dano causado à armadura;
- Rank do atacante;
- Rank da vítima;
- Lado do atacante (CT ou Terrorist);
- Lado da vítima;
- Indicador de bomba plantada;
- Região do corpo atingida.

O primeiro arquivo do conjunto de dados possui aproximadamente **5,99 milhões de registros**, ocupando cerca de **1,2 GB**. Nos experimentos finais foram utilizadas duas partes do dataset para aumentar a carga de processamento e permitir uma avaliação mais representativa do desempenho do Spark.

3. Arquitetura da Solução

Toda a aplicação foi desenvolvida utilizando Docker Compose para simular um ambiente distribuído.

A arquitetura é composta pelos seguintes componentes:

- Spark Master;
- Um ou mais Spark Workers;
- Spark Job (responsável pela execução da aplicação).

O Spark Job realiza a leitura dos arquivos CSV, executa os workloads implementados e salva os resultados obtidos em arquivos CSV. A quantidade de workers é definida durante a execução do benchmark, permitindo comparar o desempenho da aplicação utilizando diferentes níveis de paralelismo.

4. Workloads Implementados

Foram implementados três workloads com diferentes níveis de complexidade computacional:

Workload 1 – Eficiência das Armas

Agrupar todos os eventos de dano por arma e categoria da arma.

São calculadas as seguintes métricas:

- Quantidade de eventos;
- Dano médio causado à vida;
- Dano médio causado à armadura;
- Percentual de headshots.

Objetivo:

Identificar quais armas apresentam maior eficiência durante as partidas.

Workload 2 – Rank dos Jogadores versus Desempenho

Inicialmente são agregadas estatísticas individuais de cada jogador e, posteriormente, são calculadas métricas por rank.

As principais métricas calculadas são:

- Dano total causado;
- Dano médio;
- Quantidade de eventos.

Objetivo:

Verificar se jogadores de ranks mais elevados apresentam melhor desempenho em relação aos demais.

Workload 3 – Vantagem entre CT e Terrorist

Agrupar os eventos considerando:

- Lado do atacante;
- Lado da vítima;
- Situação da bomba (plantada ou não).

As métricas calculadas incluem:

- Quantidade de eventos;
- Dano médio;
- Dano total.

Objetivo:

Analisar se existe vantagem de dano para algum dos lados em diferentes situações da partida.

5. Ambiente Experimental

Os experimentos foram realizados utilizando:

- Sistema Operacional Linux;
- Docker;
- Docker Compose;
- Apache Spark 3.5.

Cada execução utilizou um cluster Docker configurado com diferentes quantidades de workers.

Foram realizados experimentos com:

- 1 Worker;
- 2 Workers;

- 4 Workers.

Cada configuração foi executada cinco vezes, permitindo calcular medidas estatísticas sobre os tempos de execução.

6. Metodologia Experimental

O objetivo dos experimentos foi analisar o impacto da quantidade de workers sobre o tempo de execução dos workloads, utilizando consultas assíncronas para permitir a execução concorrente das análises e melhor aproveitamento dos recursos disponibilizados pelo Apache Spark. Para cada configuração foram realizados os seguintes passos:

1. Inicialização do cluster Spark utilizando Docker Compose;
2. Leitura dos arquivos CSV;
3. Execução dos três workloads;
4. Registro do tempo total de leitura e execução de cada workload;
5. Repetição do experimento cinco vezes.

Ao final foram calculados:

- Média dos tempos de leitura e execução;
- Desvio padrão.

Essas métricas permitem avaliar tanto o desempenho médio quanto a estabilidade das execuções e leituras.

7. Resultados Experimentais

Tabela 1 – Tempo médio de execução dos workloads:

Workload	Workers	Média (s)	Desvio Padrão (s)
W1	1	22.39	1.69
W1	2	24.43	0.77
W1	4	23.18	0.67
W2	1	17.73	1.71
W2	2	16.71	1.63
W2	4	15.81	2.23

Workload	Workers	Média (s)	Desvio Padrão (s)
W3	1	16.13	3.68
W3	2	15.62	2.18
W3	4	14.84	2.11

Tabela 2 – Tempo médio de leitura dos arquivos CSV.

Workers	Run 1 (s)	Run 2 (s)	Run 3 (s)	Run 4 (s)	Run 5 (s)	Média (s)	Desvio Padrão (s)
1	85.120	81.570	74.762	71.180	69.821	76.49	6.63
2	79.060	78.652	75.889	76.503	82.510	78.52	2.61
4	82.952	85.953	82.016	89.380	76.609	83.38	4.76

8. Discussão dos Resultados

Os resultados obtidos demonstram que o desempenho do pipeline implementado em Apache Spark varia de acordo com as características de cada workload e com a configuração do ambiente distribuído. Conforme apresentado na Tabela 1, o aumento do número de workers não resultou em melhorias uniformes para todas as consultas. O Workload 1 apresentou seu melhor desempenho utilizando apenas 1 worker (22,39 s), enquanto as configurações com 2 e 4 workers registraram tempos médios superiores. Esse comportamento indica que o custo associado à distribuição das tarefas, comunicação entre os workers e sincronização do processamento foi maior que o benefício obtido com a paralelização, evidenciando que workloads menos intensivos podem não aproveitar eficientemente a arquitetura distribuída do Spark.

Em contrapartida, os Workloads 2 e 3 apresentaram o comportamento esperado para aplicações distribuídas. Em ambos os casos, observou-se redução gradual do tempo médio de execução à medida que o número de workers aumentou, alcançando os melhores resultados com 4 workers, que registraram 15,81 s e 14,84 s, respectivamente. Embora os ganhos absolutos tenham sido relativamente modestos, os resultados demonstram que consultas compostas por operações de agregação e processamento distribuível conseguem explorar melhor o paralelismo oferecido pelo Spark. Os desvios padrão observados entre as execuções também evidenciam certa variabilidade nos experimentos, possivelmente causada pela

inicialização da JVM, gerenciamento de memória, uso de cache e pela disputa por recursos do sistema operacional e dos containers Docker.

A Tabela 2, referente ao tempo de leitura dos arquivos CSV, apresentou um comportamento distinto daquele observado durante a execução dos workloads. A menor média foi obtida utilizando 1 worker (76,49 s), enquanto as configurações com 2 e 4 workers apresentaram tempos médios de 78,52 s e 83,38 s, respectivamente. Esses resultados indicam que a etapa de leitura é limitada principalmente pelo subsistema de armazenamento e pelo acesso aos arquivos, sendo pouco beneficiada pelo aumento do paralelismo em um ambiente com recursos restritos. Além disso, a configuração com 2 workers apresentou o menor desvio padrão (2,61 s), indicando maior estabilidade entre as execuções, enquanto a configuração com 1 worker apresentou maior variabilidade (6,63 s).

De forma geral, os experimentos confirmam que a solução proposta é capaz de processar um conjunto de dados com aproximadamente 10,5 milhões de registros em tempos inferiores a 25 segundos para todos os workloads avaliados, demonstrando a viabilidade da utilização do Apache Spark para esse tipo de aplicação. Entretanto, os resultados também evidenciam que o ganho proporcionado pela paralelização depende diretamente da natureza das operações executadas e dos recursos disponíveis no ambiente de execução. Como os testes foram realizados em uma máquina virtual Linux com aproximadamente 8 GB de memória RAM, é esperado que ambientes com maior capacidade computacional e clusters mais robustos apresentem melhor escalabilidade, menor variabilidade entre as execuções e reduções adicionais nos tempos de processamento.

9. Repositório do Projeto

O projeto foi disponibilizado em um repositório público no GitHub contendo:

- Código-fonte da aplicação;
- Scripts de benchmark;
- Documentação do projeto;
- Slides de Apresentação;

Link: https://github.com/RATZogun/Final_Project_Big_Data.git